

Practical: 3

Implementation of max-heap sort algorithm

Aim: To Implementation of max-heap sort algorithm

ALGORITHM:

Input:- Unsorted array
Output:- Sorted array
Step 1:- Start
Step 2:- Read the number of elements, size
Step 3:- Read the array elements arr[1] to arr[size]
Step 4:- Call heapsort(arr, size)
Step 5:- [Build max Heap] Repeat for $i = \frac{size}{2}$ down to 1:
 • Call maxHeapify(arr, size, i)
Step 6:- [Extraction phase] Repeat for $i = size$ down to 2
 • Swap arr[1] and arr[i]
 • Call maxHeapify(arr, i-1, 1)
Step 7:- Display sorted array arr[1] to arr[size]
Step 8:- Stop
Sub-Algorithm: maxHeapify(arr, size, i)
Step 1:- Set largest = i
Step 2:- Set l = 2*i
Step 3:- Set r = 2*i+1
Step 4:- If $l \leq size$ and $arr[l] > arr[largest]$, set largest = l
Step 5:- If $r \leq size$ and $arr[r] > arr[largest]$, set largest = r
Step 6:- If largest $\neq i$:
 • Swap arr[i] and arr[largest]
 • Call maxHeapify(arr, size, largest) (recursive)
Step 7:- Return

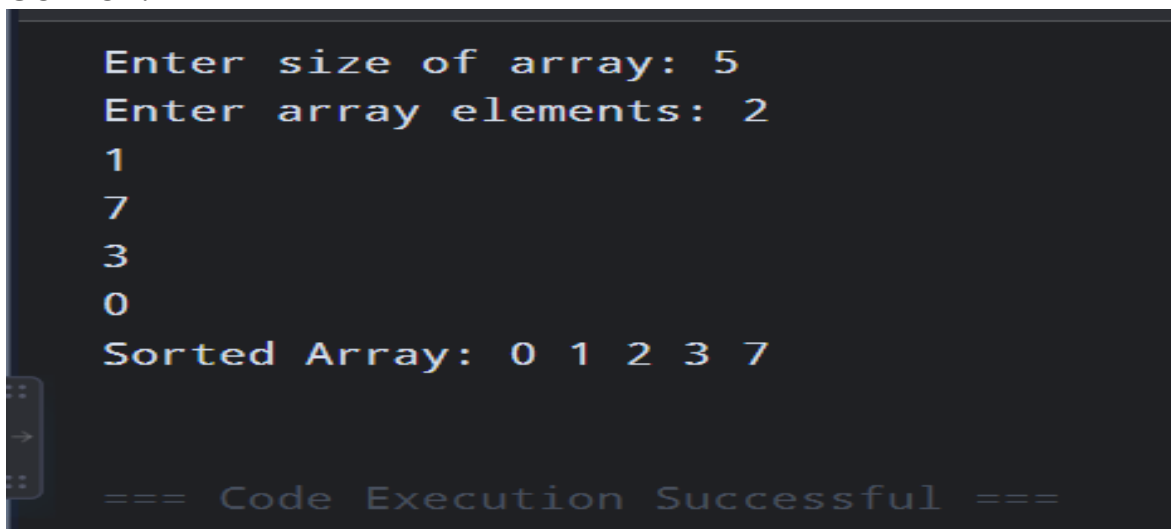
PROGRAM:

```
#include <iostream>
using namespace std;
class HeapSort
{
public:
    void maxHeapify(int arr[], int size, int i)
    {
```

```
int largest = i;
int l = (2 * i);
int r = (2 * i) + 1;
while (l <= size && arr[l] > arr[largest])
{
    largest = l;
}
while (r <= size && arr[r] > arr[largest])
{
    largest = r;
}
if (largest != i)
{
    swap(arr[i], arr[largest]);
    maxHeapify(arr, size, largest);
}
}
void heapSort(int arr[], int size)
{
    for (int i = size / 2; i >= 1; i--)
    {
        maxHeapify(arr, size, i);
    }
    for (int i = size; i >= 2; i--)
    {
        swap(arr[1], arr[i]);
        maxHeapify(arr, i - 1, 1);
    }
}
void display(int arr[], int size)
{
    cout << "Sorted Array: ";
    for (int i = 1; i <= size; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}
};
int main()
{
```

```
HeapSort h;  
int arr[100];  
int size;  
cout << "Enter size of array: ";  
cin >> size;  
cout << "Enter array elements: ";  
for (int i = 1; i <= size; i++)  
{  
    cin >> arr[i];  
}  
h.heapSort(arr, size);  
h.display(arr, size);  
return 0;  
}
```

OUTPUT:



The screenshot shows a terminal window with a dark background. The text is as follows:

```
Enter size of array: 5  
Enter array elements: 2  
1  
7  
3  
0  
Sorted Array: 0 1 2 3 7  
  
=== Code Execution Successful ===
```

TIME COMPLEXITY:



```

void maxHeapify (int arr[], int size, int i) {
    int largest = i;  $\rightarrow O(1)$ 
    int d = (2*i+1);  $\rightarrow O(1)$ 
    int r = (2*i+2);  $\rightarrow O(1)$ 
    while (d <= size && arr[d] > arr[largest]) {
        largest = d;  $\rightarrow O(1)$ 
    }
    while (r <= size && arr[r] > arr[largest]) {
        largest = r;  $\rightarrow O(1)$ 
    }
    if (largest != i)  $\rightarrow O(1)$  {
        swap (arr[i], arr[largest]);  $\rightarrow O(1)$ 
        maxHeapify (arr, size, largest);  $\rightarrow T(\dots)$ 
    }
}

```

BEST, AVERAGE, WORST CASES:

Best case

Even if the array is already sorted, your heapSort() still performs:

-for (int i = size; i >= 2; i--)

and calls maxHeapify() after every execution.

Therefore, the sorting phase still takes: $O(n \log n)$

So:

Best case = $O(n \log n)$

Average case -for an Average/Random input, the heap is built and elements are repeatedly extracted.

Build heap: $O(n)$

Extraction: $(n-1) O(\log n)$

therefore:

$T(n) = O(n) + (n-1)O(\log n)$

$T(n) = O(n \log n)$

So

Average case = $O(n \log n)$

Worst case -the worst case, maxHeapify() can travel down the height of the heap.

-for each extraction: $O(\log n)$

there are approximately n extractions:

$T(n) = n \log n$

Therefore:

worst case = $O(n \log n)$

Build heap: $O(n)$

Heap sort: $n \times O(\log n)$
= $O(n \log n)$

total: $O(n) + O(n \log n)$
= $O(n \log n)$

Space = $O(\log n)$

Conclusion:

Heap Sort was successfully implemented using Max Heapify. The algorithm sorts the elements by building a max heap and repeatedly moving the largest element to its correct position. Its best, average, and worst-case time complexity is $O(n \log n)$.